## CS6423: Scalable Computing for Data Analytics

# Optimizing Machine Learning Models: Pruning and Quantization


**Submitted By Team Yellow**

Alan George Thomas 124102586
Joel Peace Cottage Abees 124100836
Steve Ajoy 123112673
Amina Nizam 124101460
Ninglin Ou 123113268
Lakshya Rajeev Suri 124101111
Kerem Erden 124101110

School of Computer Science & Information Technology

University College, Cork


April 9, 2025

**Chapter 1 : Objective** **4**
**Chapter 2 : Background** **5**
2.1 Convolutional Neural Network 5
2.2 Models and Dataset 5
2.2.1 ResNeT50 5
2.2.2 Tiny ImageNet-200 5
2.2.3 LeNet 5
2.2.4 MNIST 6
2.3 Quantization 6
2.4 Pruning 7
2.5 Evaluation Metrics 7
**Chapter 3 : What we did** **9**
3.1 LeNet on MNIST (Small Dataset) 9
3.1.1 Model and Dataset 9
3.1.2 Baseline Training 10
3.1.3 Quantization Setup 10
3.1.4 Pruning Setup 10
3.1.5 Combined Compression 11
3.1.6 Visualization and Deployment 12
3.2 ResNet50 - ImageNet200 (Big Dataset) 12
3.2.1 Baseline Model Construction 12
3.2.2 Compression Pipeline Design 13
3.2.3 Pruning 14
3.2.4 Quantisation 15
3.3 Android Application for Real-Time Image Classification using PyTorch Mobile 16
**Chapter 4 : What we found** **18**
4.1 Findings – LeNet on MNIST 18
4.1.1 Structured Pruning Grid Search 18
4.1.2 Model Size reduction 19
4.1.3 Accuracy Comparison 20
4.1.4 Cross-Entropy Loss Comparison 21
4.1.5 Inference Speed 22
4.1.6 Per-Class Accuracy 23
4.2 Findings – ResNet-50 on Tiny ImageNet-200 23
4.2.1 Baseline Performance (Before Compression) 24
4.2.2 Unstructured Pruning Findings 25
4.2.3 Structured Pruning Findings 26
4.2.4 Iterative Pruning Findings 27

4.2.5 Comparison of All Pruning Strategies 28
4.2.6 Quantization Findings 29
**Chapter 5 : Future Scope** **31**
**Chapter 6 : Conclusion** **32**
6.1 Key Takeaways: Comparing LeNet and ResNet-50 Under Compression 32
6.2 Final Reflection 32

# Chapter 1 : Objective

- Apply two model compression techniques — **quantization** and **pruning** — on two model-dataset combinations
    - **LeNet** on **MNIST** (small-scale setup)
    - **ResNet50** on **ImageNet200** (large-scale setup)
- Experiment with different types of quantization
    - Dynamic quantization (per-tensor symmetric, per-channel symmetric)
    - Static quantization (per-tensor symmetric, per-channel symmetric)
- Experiment with different types of pruning
    - Unstructured pruning: Magnitude-Based Pruning, Gradient-Based Pruning
    - Structured pruning
    - Iterative pruning
- Compare the performance of these techniques using key evaluation metrics:
    - Top-1 and Top-5 Accuracy
    - Categorical Cross-Entropy Loss
    - Model Size
    - Inference Time
- Assess the impact of compression on model performance for both large and small datasets. Draw inferences and conclusions from the data gathered.
- Deploy the compressed models (ResNet50 and LeNet) onto mobile devices and test the compressed model on unseen real world data to get a sense of accuracy and inference time.

# Chapter 2 : Background

This section provides a brief overview of the core concepts, datasets, models, compression techniques, and evaluation metrics used in this project.

## 2.1 Convolutional Neural Network

Convolutional Neural Networks are a class of deep neural networks predominantly used in computer vision tasks. CNNs consist of multiple layers, including convolutional layers, pooling layers, and fully connected layers, which help in learning spatial hierarchies of features from input data. However, the deep structure of CNNs can result in high memory usage and computational latency, making them inefficient for deployment on edge devices without optimization.

## 2.2 Models and Dataset

### 2.2.1 ResNeT50

ResNet-50 is a widely adopted deep convolutional neural network architecture introduced by He et al. in 2015. It consists of 50 layers and is built using residual blocks that incorporate shortcut connections to enable more effective gradient flow during training. This residual learning framework allows the model to be significantly deeper without suffering from degradation problems common in traditional architectures. ResNet-50 uses a bottleneck design to reduce computational cost while maintaining performance. Its balance of depth, accuracy, and efficiency has made it a standard backbone for a variety of image classification and computer vision tasks.

### 2.2.2 Tiny ImageNet-200

Tiny ImageNet-200 is a lightweight image classification dataset derived from the larger ImageNet dataset. It contains 200 object categories, with each category having 500 training images, 50 validation images, and 50 test images. All images are resized to 64×64 pixels, significantly reducing the computational requirements compared to the original ImageNet. Despite its smaller scale, Tiny ImageNet retains much of the complexity and diversity of real-world image data, making it a popular benchmark for evaluating deep learning models and exploring model compression techniques such as pruning and quantization.

### 2.2.3 LeNet

LeNet-5 is a pioneering CNN architecture designed by Yann LeCun for digit recognition tasks. It consists of 7 layers, including convolutional and subsampling layers, and is well-suited for

small-scale datasets such as MNIST. Due to its simplicity and relatively small size, LeNet-5 is often used in early-stage experimentation with compression techniques.

### 2.2.4 MNIST

MNIST (Modified National Institute of Standards and Technology) is a classic dataset used for handwritten digit recognition. It contains 60,000 training and 10,000 test grayscale images of 28×28 pixels. Due to its simplicity, it is frequently used as a benchmark for initial model testing and performance validation.

## 2.3 Quantization

Quantization is a model compression technique aimed at reducing numerical precision in neural networks, thereby decreasing model size and improving computational efficiency. This is achieved by converting high-precision floating-point numbers (e.g., Float32) into lower-precision integers (e.g., Int8) using the formula:

$$q = round(\frac{r}{S} + Z)$$

where:
q: Quantized value (e.g., int8)
r: Original floating-point value
S: Scaling factor = $\frac{max(r) - min(r)}{max(Q) - min(Q)}$
Z: Zero point = $round\left(-\frac{min(r)}{S} + min(Q)\right)$

Various quantization techniques we used in the project are mentioned below.

- **Dynamic Quantization:** Quantized weights ahead of time but quantizes activations dynamically during inference, making it suitable for models with many linear layers. This is because PyTorch's dynamic quantization only quantizes the linear and LSTM layers and doesn't handle convolution layers.
- **Static Quantization:** Pre-quantizes both weights and activations using a calibration dataset to compute scale and zero-point values, offering better compression for models with convolutional layers. Convolution layers are handled in PyTorch's implementation of static quantization.
- **Symmetric Quantization:** The range of quantized values is centered around zero, simplifying calculations but less effective for asymmetric data distributions.

- **Quantization Granularity**
    - **Per-Tensor Quantization:** Applies a single scale and zero-point across an entire tensor. It is simple but may reduce accuracy in models with varied channel statistics. Involved less meta-deta storage like scale and zero point values.
    - **Per-Channel Quantization:** Calculates scale and zero-point for each channel independently, improving accuracy for convolutional layers at the cost of added complexity. Involves more meta-data storage like scale and zero-point values.

## 2.4 Pruning

Pruning is a structural compression technique that involves removing less significant weights or neurons from a neural network. This results in a sparser model with reduced complexity, faster inference, and lower memory usage. Various pruning techniques we used in the project are mentioned below.

- **Unstructured Pruning**: Removes individual weights independently, creating sparse models that require specialized hardware for efficient inference.
    - **Magnitude-Based Pruning**: Eliminates weights with the smallest magnitudes, offering a simple and effective way to reduce model complexity.
    - **Gradient-Based Pruning**: Uses gradient information to identify and prune less critical weights, potentially preserving more important connections.
- **Structured Pruning**: Removes entire neurons, channels, or layers, maintaining dense operations for better hardware compatibility but with less granularity.
- **Iterative Pruning**: Prunes the model gradually over multiple training cycles, allowing recovery of accuracy between steps for improved final performance.

## 2.5 Evaluation Metrics

Following metrics are used in this project.

- **Top-1 Accuracy**: Measures how often the model's top prediction matches the true label. This metric is crucial for evaluating the overall effectiveness of the model after compression.
- **Top-5 Accuracy**: Measures how often the correct label is within the model's top five predictions. This is useful for tasks with a large number of classes where minor ranking shifts may still indicate good performance.
- **Categorical Cross-Entropy (CCE)**: Measures the model's confidence in making correct predictions. A lower confidence results in a higher loss. Even if accuracy remains stable, CCE can reveal how confident the model is after compression.

- **Model Size**: Represents the memory footprint of the model. Compression techniques aim to reduce model size while maintaining performance, helping to analyze trade-offs between size and accuracy.
- **Inference Time**: The time required for the model to process a batch of inputs. Faster inference is critical for real-time applications, and this metric helps assess the efficiency improvements gained through compression techniques.

# Chapter 3 : What we did

To manage the scope of the project effectively, we split our group into two teams. One team focused on applying compression techniques to a small-scale setup using the LeNet model on the MNIST dataset, while the other team worked on a large-scale setup using the ResNet50 model on the ImageNet dataset.

We had a total of **three weeks** to complete the project.

- **Week 1**: Focused on understanding the requirements, finalizing tools, and dividing tasks among team members.
- **Week 2**: Carried out the implementation and applied the compression techniques.
- **Week 3**: Conducted additional experiments, gathered results, and prepared the analysis for the report.

In this section, we describe the specific steps each team followed, the tools and libraries used, how compression techniques were implemented, and how the models were prepared for deployment and evaluation.

## 3.1 LeNet on MNIST (Small Dataset)

### 3.1.1 Model and Dataset

We manually implemented the LeNet architecture in PyTorch as it is not available as a pre-defined model in the PyTorch library. The Model was defined using `nn.Sequential`, and included two convolutional layers followed by average pooling, and three fully connected layers ending in a softmax output. The activation function used throughout the network was ReLU.

For the dataset, we used the MNIST dataset from the `torchvision.datasets` module. We applied the following preprocessing steps using `torchvision.transforms`:

- Converted images to PyTorch tensors using `ToTensor()`
- Normalized the pixel values to have a mean of 0.1307 and a standard deviation of 0.3081, which are standard values for MNIST

The dataset was split into training and testing sets using PyTorch's built-in `train=True/False` flags. The training set contained 60,000 images and the test set contained 10,000 images. We used a `DataLoader` to batch the data and shuffle the training set to ensure better generalization during training.

### 3.1.2 Baseline Training

The original LeNet model was trained using PyTorch. We used the Adam optimizer with a learning rate of 0.001 and trained for 10 epochs with a batch size of 64. The loss function was categorical cross-entropy, appropriate for multi-class classification problems. After training, we evaluated the model's performance on the test set and recorded accuracy, cross-entropy loss, inference time, and model size. This served as the baseline against which compressed versions were compared.

### 3.1.3 Quantization Setup

We applied quantization to reduce model size and improve inference speed, using both **dynamic** and **static** quantization techniques provided by PyTorch.

**Static Quantization**:

- Required additional model preparation using `torch.quantization.prepare` and `convert`.
- Performed calibration on a sample batch from the training data to collect activation statistics.
- Quantized both weights and activations to int8.
- Involved rewriting the model to insert `QuantStub` and `DeQuantStub` modules for quantization flow.

**Dynamic Quantization**:

- Applied to the `Linear` layers using `torch.quantization.quantize_dynamic`.
- We applied dynamic quantization using int8 on the fully connected layers.
- Did not require retraining and was quick to implement.

After quantizing both types, we saved the model and measured model size, inference time and accuracy.

### 3.1.4 Pruning Setup

To reduce the number of parameters in the LeNet model and explore the impact of sparsity, we applied both **unstructured** and **structured pruning**. This allowed us to compare fine-grained and coarse-grained pruning approaches in terms of size reduction, accuracy, and inference time.

We used two different libraries for the pruning tasks:

- **PyTorch's `torch.nn.utils.prune`** for unstructured pruning
- **Torch-Pruning (`torch-pruning` or `tp`)** for structured pruning

**Unstructured Pruning:**

- Implemented using `prune.l1_unstructured` from `torch.nn.utils.prune`.
- Applied to individual weights in both convolutional (`Conv2d`) and fully connected (`Linear`) layers.
- Weights with the smallest absolute values were zeroed out.
- A target **sparsity of 30%** was applied layer-wise.
- This approach does not remove weights from memory but masks them internally.

**Structured Pruning:**

- Used the Torch-Pruning library (`tp`) to prune entire filters (channels) in convolutional layers and neurons in fully connected layers. Torch-Pruning is an open-source library available at: https://github.com/VainF/Torch-Pruning.
- Structured pruning was tested as part of a **grid search**, where we experimented with the magnitude method and the taylor method.
- We tried pruning on three layers, convolutional , fully connected, and both.
- Results of the gridsearch are discussed in chapter 4.
- After pruning each model was retrained to recover performance.
- Structured pruning results in actual changes to the model architecture, leading to more efficient execution on hardware compared to unstructured pruning, which only masks weights.

### 3.1.5 Combined Compression

A common real world approach is to combine pruning and quantisation. We chose structured pruning for this combination because that gave us the most size reduction. Both static and dynamic quantisation versions were created and evaluated.

We chose to do pruning first because it operates on full-precision weights, allowing more accurate importance estimation (e.g., using weight magnitude or gradients). Performing quantization first would constrain weight representation (e.g., to int8), which can reduce pruning effectiveness. This order also aligns with best practices from literature to preserve accuracy during compression.

Although **Quantization-Aware Pruning (QAP)**—where both pruning and quantization are considered jointly—has been proposed to optimize this trade-off even further (e.g., Wang et al., 2021), our focus remained on a modular pipeline where each compression step could be independently evaluated and tuned. This sequential strategy ensured easier debugging, clearer insights, and compatibility with PyTorch's static quantization flow.

The combined model was then exported and evaluated on our evaluation metrics.

### 3.1.6 Visualization and Deployment

We used **Matplotlib** to visualize the following:

- Training vs. validation accuracy over epochs
- Training vs. validation loss
- Comparison of model sizes (original, pruned, quantized, combined)
- Comparison of inference times across different versions

Conclusions drawn from the data are discussed in chapter 4.

Finally, we exported the compressed models using **TorchScript** via `torch.jit.trace`, which converts the PyTorch model into a form suitable for deployment. We performed live predictions using new, unseen digit images captured from the phone.

## 3.2 ResNet50 - ImageNet200 (Big Dataset)

This section is about what we did using ResNet-50 on Tiny ImageNet200. The specifications of the Model(ResNet-50) and DataSet(Tiny ImageNet200) are discussed in chapter 2.

### 3.2.1 Baseline Model Construction

ResNet50 was originally designed and pre-trained for the ImageNet-1000 dataset, which contains 1000 classes and higher resolution images. We first needed to adapt this pretrained model so it is compatible with the ImageNet200 data set which contained 200 classes and lower resolution images.

The pretrained ResNet-50 model was adapted for the Tiny ImageNet-200 dataset by:

- Replacing the original classification head with a new fully connected layer with 200 output units.
- To retain the benefits of transfer learning while preventing early overfitting, we **froze the earlier convolutional layers** and fine-tuned only the **final three convolutional layers** of the network.

Given the relatively small dataset size, we were concerned with overfitting. We applied several **regularization techniques**:

- Data augmentation : Random Horizontal Flipping, Random Cropping and Color Jittering.
- **L2 weight decay** to penalize large weights.
- A **dropout layer** with a rate of **0.4** before the final classification layer.

The final baseline model performance metrics are discussed in chapter 4.

## 3.2.2 Compression Pipeline Design

The compression process was organized as a sequential pipeline as shown in Figure 1 with three stages:

1. Training the baseline model
2. Applying pruning (both structured and unstructured) during fine-tuning
3. Applying post-training quantization

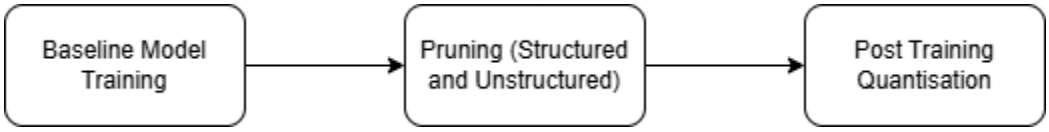


Figure 1: Compression Pipeline

This sequential design was motivated by the need to manage structural and numerical changes in a controlled and effective manner.

Applying pruning first ensured that the less important weights are set to zero while still retaining the float36 space. Applying pruning first allowed us to fine-tune the sparse model to recover any accuracy loss while avoiding the compounded complexity of quantization error.

Once the desired sparsity level is achieved (e.g., 50% zeroed weights) and the model is retrained to restore performance, we proceed with post-training quantization.

This converted the weights and activations into 8-bit integer representations, further reducing the model size and inference cost. 8-bit is a natural fit because zero values are precisely representable in int8, and the remaining non-zero weights are simply scaled. Applying quantization after pruning also ensured that calibration and scale adjustments occurred on a smaller, optimized model.

This pipeline leverages the benefits of both techniques:

- **Sparsity** (from pruning) reduces model size and computational cost.
- **Quantization** improves compatibility with hardware accelerators and further compresses the model.

Together, they allow for efficient deployment of deep learning models on resource-constrained environments.

(*Note: Detailed performance insights and outcomes will be presented in Chapter 4.*)

### 3.2.3 Pruning

Upon reviewing the architecture of ResNet-50, we noted its inherent skip connections, which pose additional challenges for pruning. To explore the effectiveness of different pruning strategies, we evaluated both structured and unstructured pruning techniques. We began with one-shot structured pruning due to its simplicity, and later extended our pipeline to iterative pruning.

**Unstructured Pruning:**

We investigated two unstructured pruning strategies:

- **Magnitude-based pruning** at 30% and 50% sparsity.
- **Gradient-based pruning** at 20%, 30%, and 50% sparsity.

All pruned models underwent a **post-pruning fine-tuning phase** to recover accuracy.

**Structured Pruning:**

We explored structured pruning at sparsity levels of 20%, 30%, and 50%. To mitigate the risk of drastic performance degradation, we froze the initial convolutional block, the first layer of each residual block, and the fully connected (FC) layer, as these components are particularly sensitive to channel-wise pruning. Preserving these key layers helps retain essential representational capacity and prevents significant accuracy loss.

**Iterative Pruning:**

Our iterative pruning pipeline consists of two main stages:

1. **Structured pruning** at 30% sparsity, followed by fine-tuning.
2. **Unstructured gradient-based pruning**, followed by another fine-tuning stage.

This two-stage process constituted one full iteration. We initially performed one iteration and monitored validation performance to determine whether additional iterations were necessary.

The rationale for this approach was to **maximize the benefit of gradient-based pruning**, which performs better when applied to a well-tuned model. After the structured pruning and fine-tuning, we tested unstructured pruning at **20% and 40% sparsity levels**.

Early stopping was implemented during the second fine-tuning stage to prevent overfitting. The validation accuracy peaked after the first epoch, and subsequent training was halted accordingly.

### 3.2.4 Quantisation

After pruning and fine-tuning, we applied **8-bit integer quantization** to the pruned ResNet-50 model. This was the final step in the compression pipeline. Quantization was applied using **PyTorch's quantization utilities**, and both **dynamic** and **static quantization** approaches were explored.

**Dynamic Quantization :**

In dynamic quantization, only the model weights are quantized, while activations remain in floating-point format during inference and are quantized during runtime. This method is best suited for models that are **fully connected layer** heavy.

- In PyTorch, dynamic quantization is limited to `Linear` layers only.
- Since ResNet-50 has only one linear layer (the final FC layer), dynamic quantization led to low size reduction.

**Static Quantization:**

Static quantization quantizes both weights and activations to int8 format. Unlike dynamic quantization, it requires calibration data. The model is run over a small subset of the training data to collect activation statistics and determine scale and zero-point values.

- In PyTorch, static quantization supports both `Conv2d` and `Linear` layers.
- The calibration set was taken from the training dataset and used to compute quantization parameters.
- This approach yielded a significant reduction in model size and increase in the CPU-computation speed, as ResNet-50 has a large number of convolutional layers.

**Quantization Scheme:**

The default scheme provided by PyTorch for both static and dynamic quantisation is per-tensor symmetric. In this scheme, a **single scale and zero-point** value is used for the entire tensor. ResNet50 contains many convolutional layers, each with multiple channels. If we use a single scaling factor per tensor, i.e. for all the weights in all channels of a layer, we lose precision in the quantised weights and activations. We soon found out we had poor accuracy due to this.

To address this issue, we switched to per-channel symmetric quantisation, in which the scale and the zero point are calculated separately for each channel of a convolutional layer weight tensor. Although this requires more computations while quantizing the model using a calibration dataset (as convolution layers can only be quantized using static quantization), the boost in the inference speed and accuracy make this extra step worth it.

## 3.3 Android Application for Real-Time Image Classification using PyTorch Mobile

Developed an Android application which enables real-time image classification using deep learning models integrated via PyTorch Mobile. It supports dynamic switching between the previously trained models - LeNet and ResNet - allowing users to compare model performance in terms of prediction and inference speed directly on an Android device.

The app provides a simple and intuitive interface. Users can select a model through radio buttons, capture an image using the device's camera, and receive instant classification results along with the model's inference time in milliseconds. The classification label is displayed on-screen, and an image preview is shown for clarity.

Key Features:

- Model Switching: Users can choose between LeNet and ResNet at runtime without needing to restart the app.
- Real-Time Camera Integration: The application allows users to capture an image from the camera, preprocess it into a suitable format (3-channel RGB, resized to 28×28), and run inference.
- Performance Metrics: The app calculates and displays inference time in milliseconds to help users compare latency between models.
- PyTorch Integration: Models are loaded as .pt files using PyTorch Android API (Module.load()), ensuring compatibility and efficient execution on mobile devices.
- Efficient Preprocessing: Bitmap images are converted into tensors with proper normalization and resizing for model compatibility.

This application showcases the feasibility of deploying and switching between multiple deep learning models on a mobile platform. Figure 2 shows the screenshots of the application.

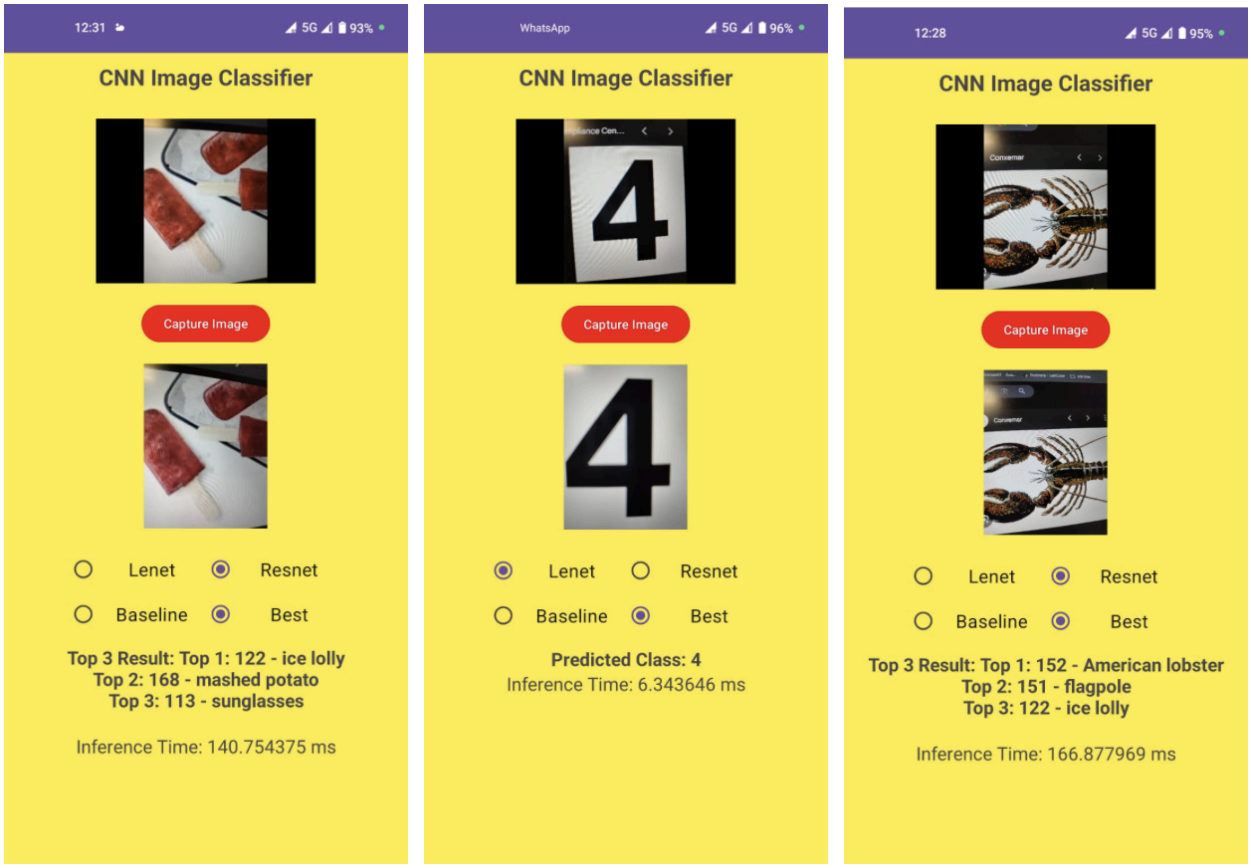


Figure 2: Application Screenshots

# Chapter 4 : What we found

This chapter presents the key findings from our experiments across both the model-dataset setups. Rather than simply reporting results, we aim to explain why each compression technique behaved the way it did.

The section is divided into two parts : One covering LeNet on MNIST and the other focusing on ResNet-50 on Tiny ImageNet-200. Each subsection discusses accuracy, loss, inference time, and model size, supported by plotted results and printed metrics.

## 4.1 Findings – LeNet on MNIST

In this section, we analyze the performance of various compression strategies applied to LeNet on the MNIST dataset.

### 4.1.1 Structured Pruning Grid Search

We began our compression experiments with a **grid search over structured pruning configurations**. The goal was to identify combinations of method, scope, and sparsity that offered meaningful compression without significantly harming performance.

We evaluated two pruning methods: Magnitude-based pruning  and Taylor-based pruning Each method was applied under three layer scopes: Only Convolutional Layers, Only Fully Connected Layers, and Both Convolutional Layers and Fully connected Layers.

To ensure the pruned models remained usable, we filtered results to include only those configurations where the **accuracy drop was no more than 0.02** (2%) compared to the baseline model. The following table shows two of the best-performing configurations from this search.

| **Structure** | **Method** | **Scope** | **Sparsity** | **Size(MB)** | **Accuracy(T1)** | **Loss(CEL)** |
|---|---|---|---|---|---|---|
| Structured | Magnitude | FC | 0.3 | 0.117 | 97.46 | 0.0825 |
| Structured | Magnitude | FC | 0.4 | 0.100 | 96.51 | 0.1293 |

These results suggest that moderate pruning (around 30% sparsity) to fully connected layers offers strong compression benefits with minimal impact on accuracy.

### 4.1.2 Model Size reduction

**What the plot shows:** Figure 3 shows the size (in KB) of each model variant, with percentage reduction compared to baseline.

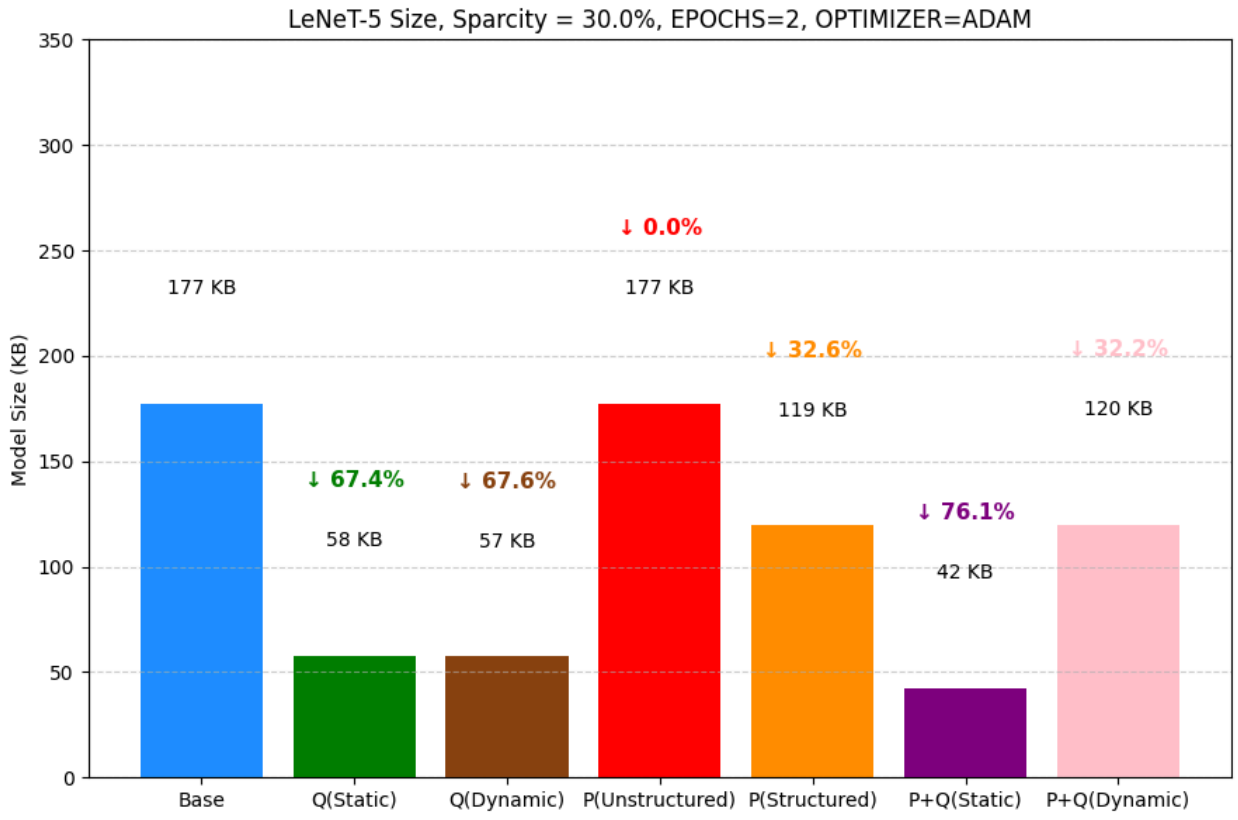


Figure 3: Model Size Comparison

**What stands out:** Structured Pruning with Static Quantisation shows a dramatic reduction (76%) in model size.

**Why it behaves that way:** Structured pruning removes entire channels or neurons from the network, leading to actual reductions in weight tensor dimensions. This directly impacts the number of parameters and memory footprint. Static quantization further compresses the model by converting both weights and activations to 8-bit integers during inference. Unlike dynamic quantization, static quantization quantizes convolutional layers and is applied ahead of inference with calibration data. The combination of these two methods compounds the compression effect, resulting in a much smaller model that still preserves structural integrity and inference capability.

### 4.1.3 Accuracy Comparison

**What the plot shows:** Figure 4 is a bar chart comparing Top-1 and Top-2 accuracy across different versions of the model: baseline, quantized (static and dynamic), pruned (structured and unstructured), and combinations of pruning + quantization.

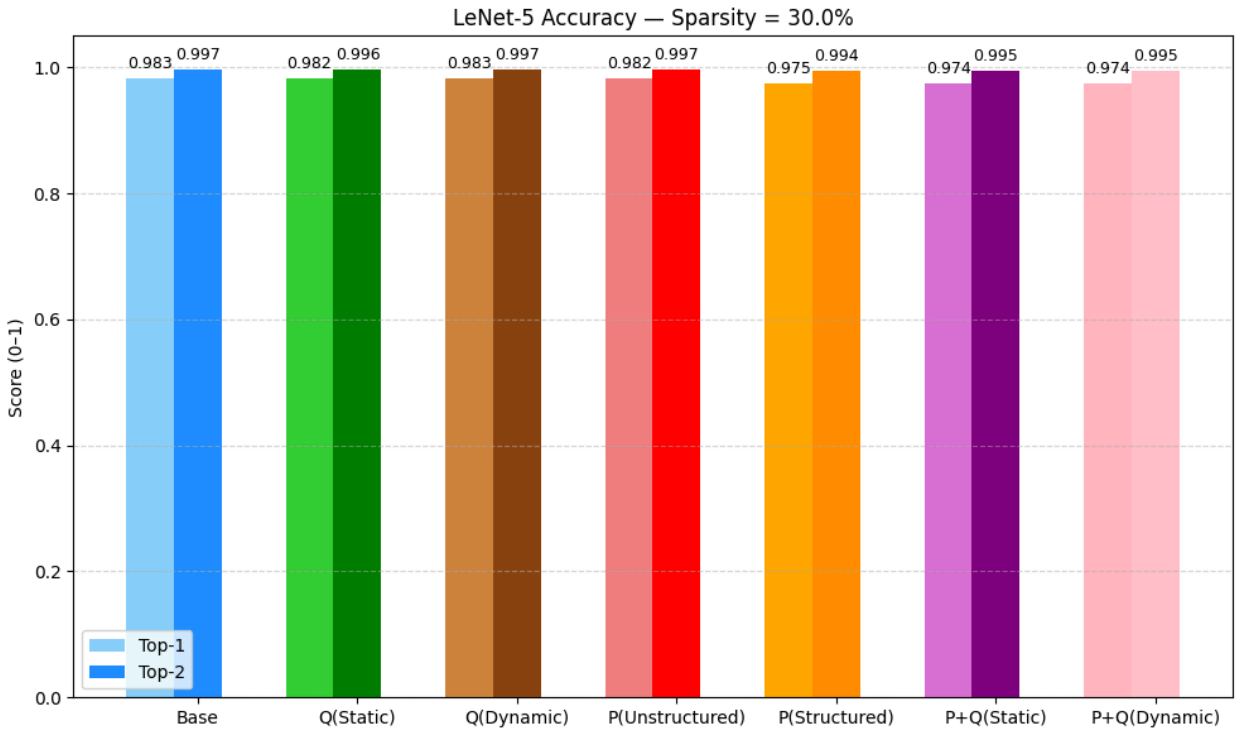


Figure 4: Accuracy Comparison

**What stands out** : There is not a lot of drop in accuracy in any of the versions. Pruning Static Quantisation still gave a 97.4% for Top-1 and 99.5% for Top-2 Accuracy despite the 76% drop in size.

**Why it behaves that way**: We performed extensive grid search across different sparsity levels, pruning scopes, and quantization types to identify configurations that yield the highest compression with minimal accuracy drop. In addition, we ran several heuristic experiments to analyze the sensitivity of accuracy to structured and unstructured pruning combined with both static and dynamic quantization. The final model configurations reflect the best trade-offs identified during these tests. This systematic exploration helped ensure that accuracy remained consistently strong across all variants, even under aggressive compression.

### 4.1.4 Cross-Entropy Loss Comparison

**What the plot shows:** Figure 5 shows a bar chart comparing average cross-entropy loss for the same model variants as above.

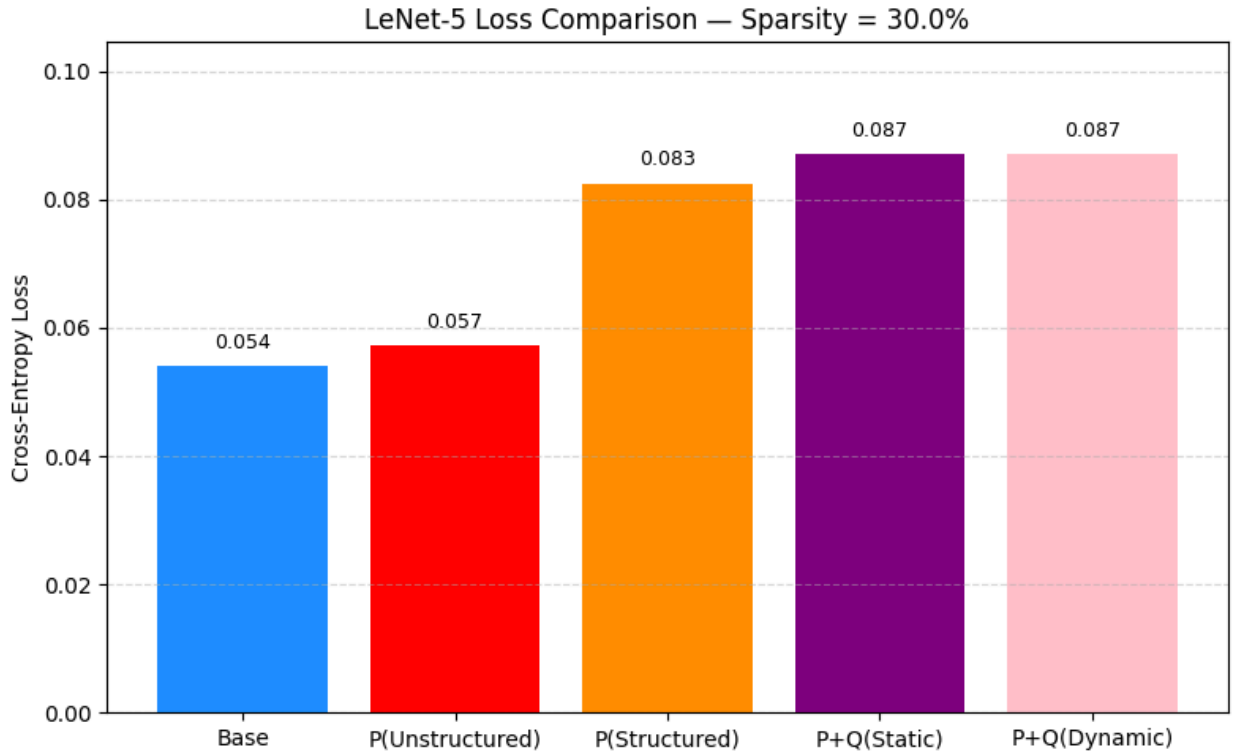


Figure 5: Cross-Entropy Loss Comparison

**What stands out**: Loss increases slightly for pruned and quantized models, especially for the pruned + quantized combinations.

**Why it behaves that way**: Both pruning and quantization introduce approximations into the model's parameters and activations. Structured pruning reduces entire channels or neurons, potentially removing some task-relevant features, while quantization lowers numerical precision—especially in activations and weights—resulting in less expressive capacity. When combined, these effects amplify, leading to a slight increase in average cross-entropy loss. The increase is more visible in the pruned + quantized configurations due to compounded approximation error. However, this trade-off is acceptable given the significant reduction in model size and only marginal impact on overall accuracy.

## 4.1.5 Inference Speed

**What the plot shows:** Figure 6 shows average inference time (in milliseconds) per input sample for each model.

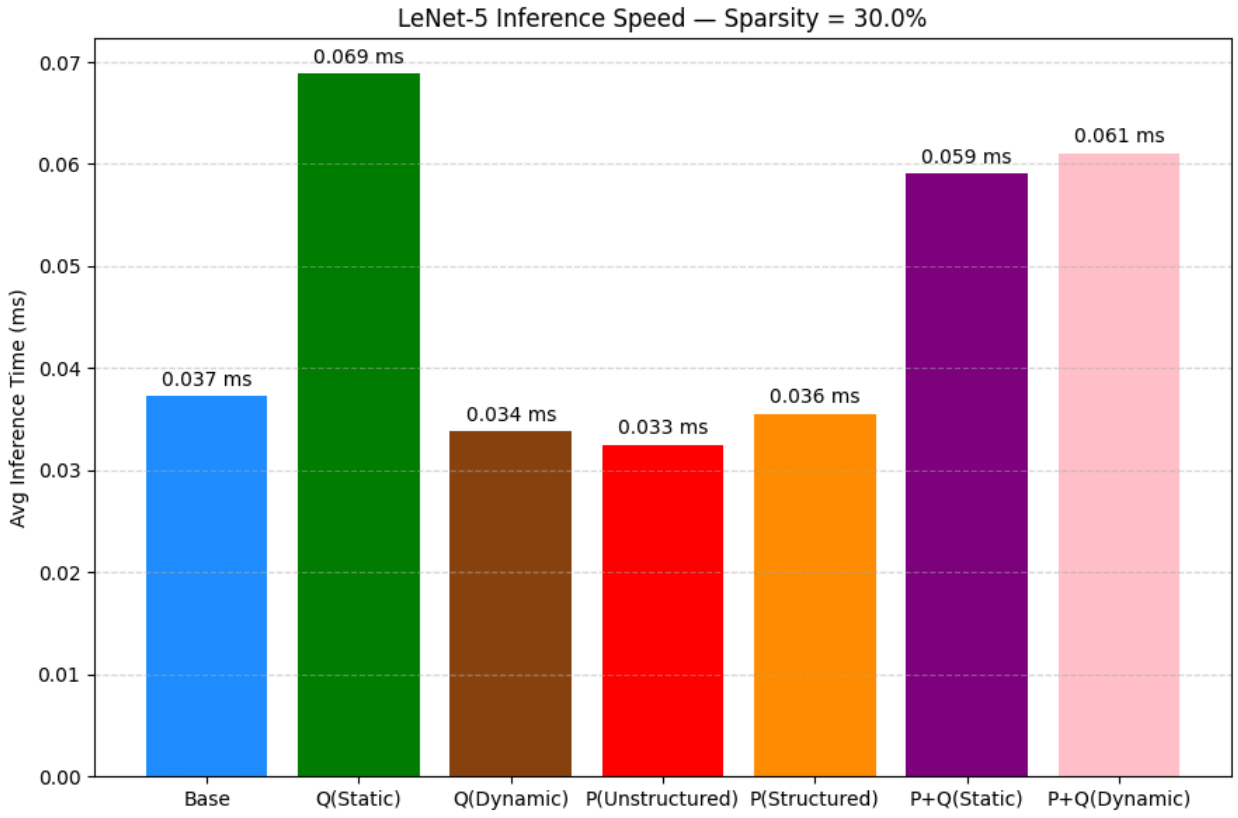


Figure 6: Inference Speed Comparison

**What stands out**: Static Quantisation has the most noticeable speedup. Pruning + Quantisation combinations had the best overall performance.

**Why it behaves that way:** Inference speed is influenced by both the number of operations and the hardware efficiency of executing them. Static quantization achieves the highest speedup because it replaces float operations with efficient 8-bit integer operations using optimized kernels (like FBGEMM), especially for convolutions and fully connected layers. Pruning reduces the model size by removing redundant channels or neurons, thus reducing computational load. The combined approach (pruning + quantization) benefits from both structural sparsity and low-precision arithmetic, leading to improved performance. However, dynamic quantization is less effective on convolution-heavy models like LeNet-5, as it only quantizes linear layers during inference. ***The 8-bit integer representation hits the sweet spot for quantization—offering a strong balance between speed and accuracy—as consistently supported in literature and practical deployments.***

### 4.1.6 Per-Class Accuracy

**What the plot shows**: Figure 7 shows how well the compressed models handle different digits. We analyzed their Top-1 accuracy for each class (digit 0–9).

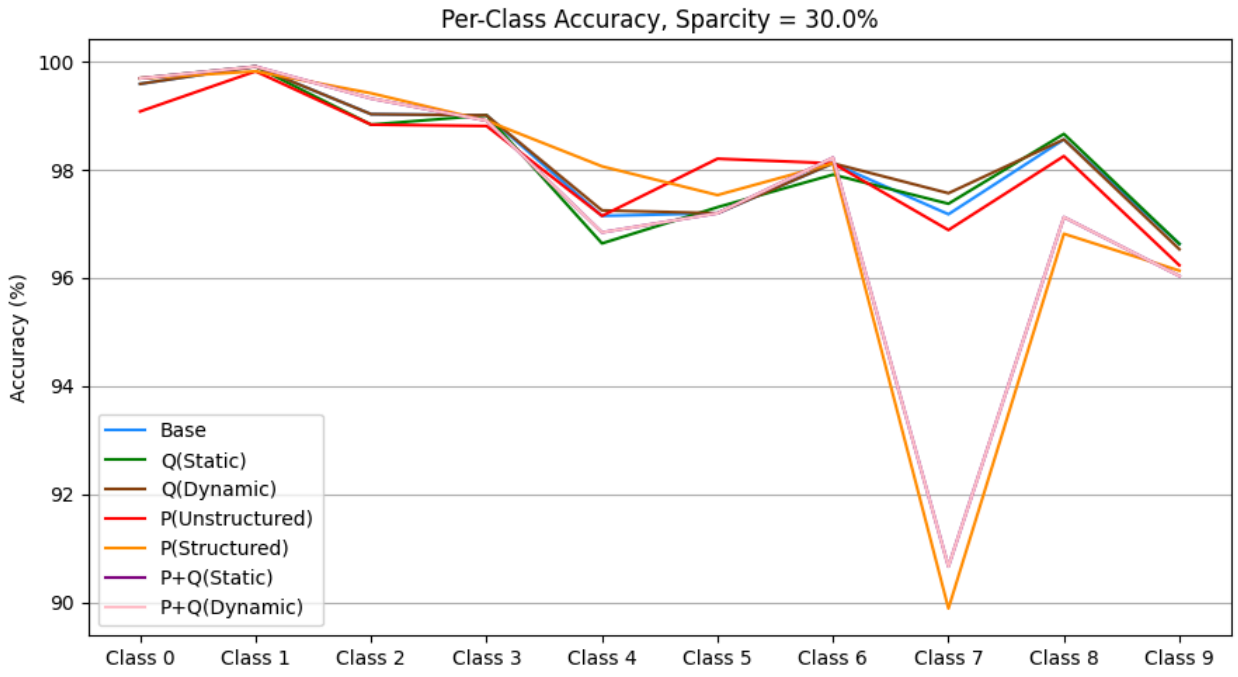


Figure 7: Per Class Accuracy Comparison

**What stands out**: Digits like ‘4’, ‘7’, and ‘9’ consistently show lower accuracy across models.

**Why it behaves that way:** Digits like ‘4’, ‘7’, and ‘9’ tend to have more visually similar features and structural variations in handwriting, making them harder to distinguish even for full-precision models. Compression techniques like pruning and quantization can further degrade feature representations, especially in deeper layers, leading to a higher chance of misclassification for these classes. This sensitivity is amplified in pruned+quantized models due to the combined effects of reduced channel capacity and lower numerical precision.

## 4.2 Findings – ResNet-50 on Tiny ImageNet-200

This section presents the evaluation results of applying pruning and quantization techniques to ResNet-50 trained on the Tiny ImageNet-200 dataset. Unlike LeNet on MNIST, ResNet-50 represents a large-scale, deep architecture, making it a more complex candidate for compression. The findings are organized to reflect how each method affected model accuracy, size, and inference time, with a focus on identifying configurations that offer the best balance between performance and efficiency.

## 4.2.1 Baseline Performance (Before Compression)

To improve generalization and ensure robustness during future pruning and fine-tuning stages, we applied data augmentation techniques such as random horizontal flipping, cropping, and color jittering.

**What the plots show**: Figure 8 shows the comparison of accuracy and loss before and after data augmentation and regularization.

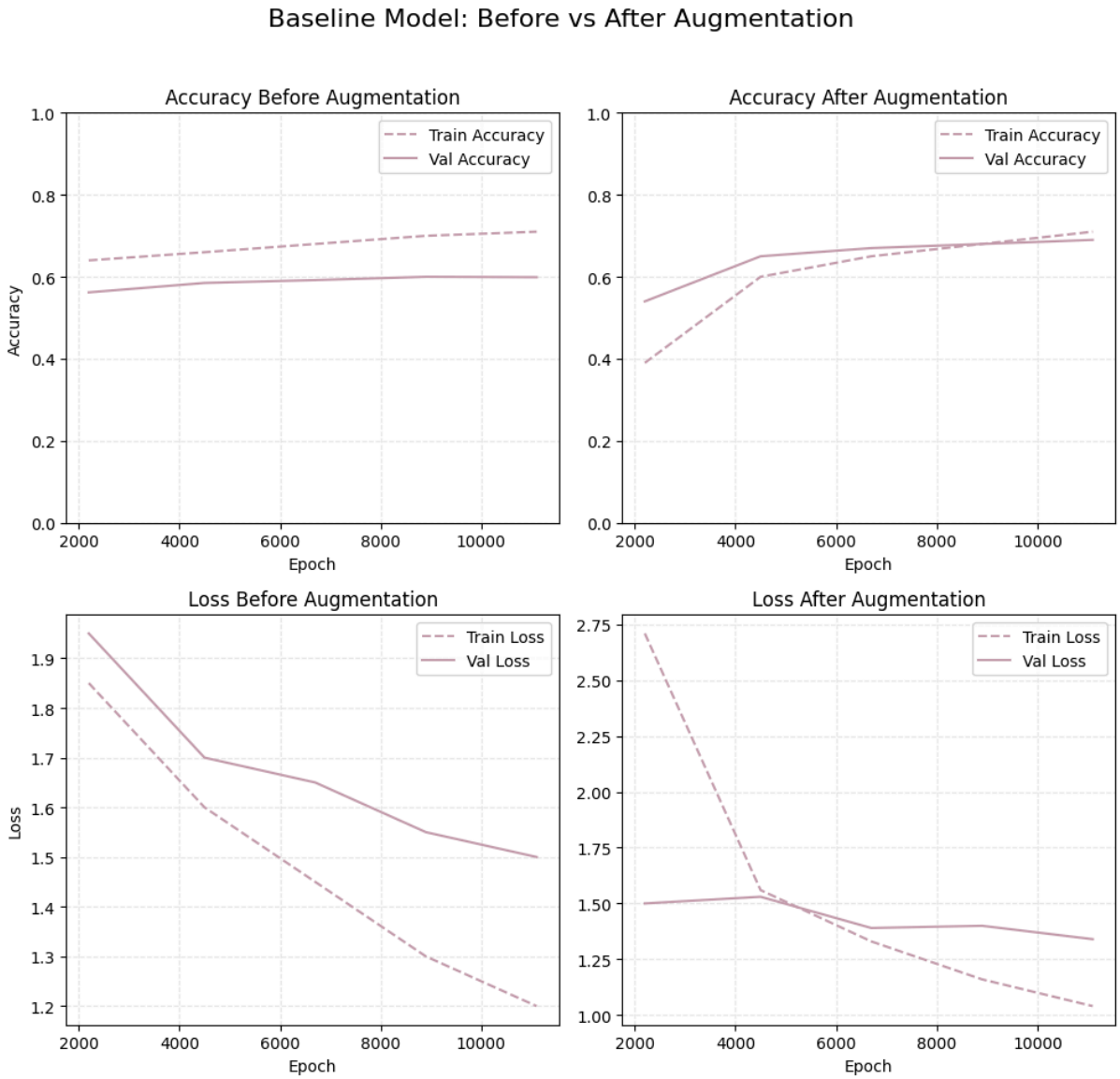


Figure 8: Before vs After Augmentation

**What stands out** : Augmentation significantly delayed overfitting, improving both training stability and validation performance. To further regularize the model, we incorporated L2 weight decay and set a dropout rate of 0.4 before the final classification layer.

Figure 9 shows the finalized Top-1 and Top-5 accuracy, and cross-entropy loss for the ResNet-50 baseline model after training.

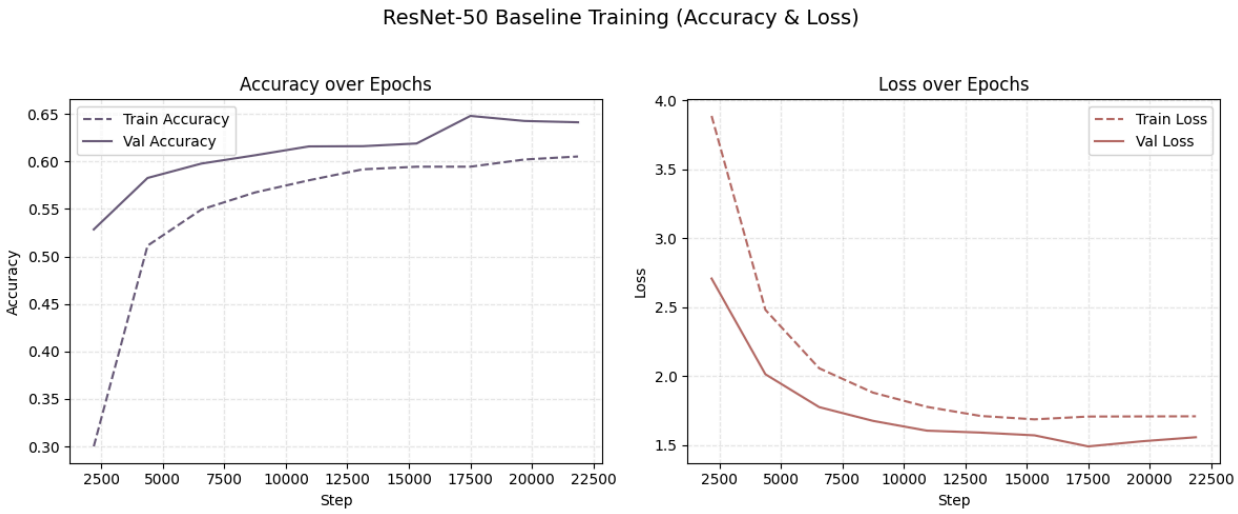


Figure 9: Baseline Accuracy and Loss

**What stands out**: The finalized baseline model achieved a top-1 accuracy of 62.43%, top-5 accuracy of 84.29%, categorical cross-entropy loss of 1.63, and a total model size of approximately 95.99 MB.

## 4.2.2 Unstructured Pruning Findings

We experimented with both magnitude-based and gradient-based pruning at multiple sparsity levels. Each configuration was followed by fine-tuning to recover any loss in performance. Unlike structured pruning, unstructured pruning removes individual weights without altering the network's overall structure.

**What the graph shows:** The first two graphs in figure 11 shows Top-1 Accuracy and Top-5 Accuracy for the two pruning methods at various sparsity levels. The third graph shows Inference Time per sample (milliseconds) for each configuration.

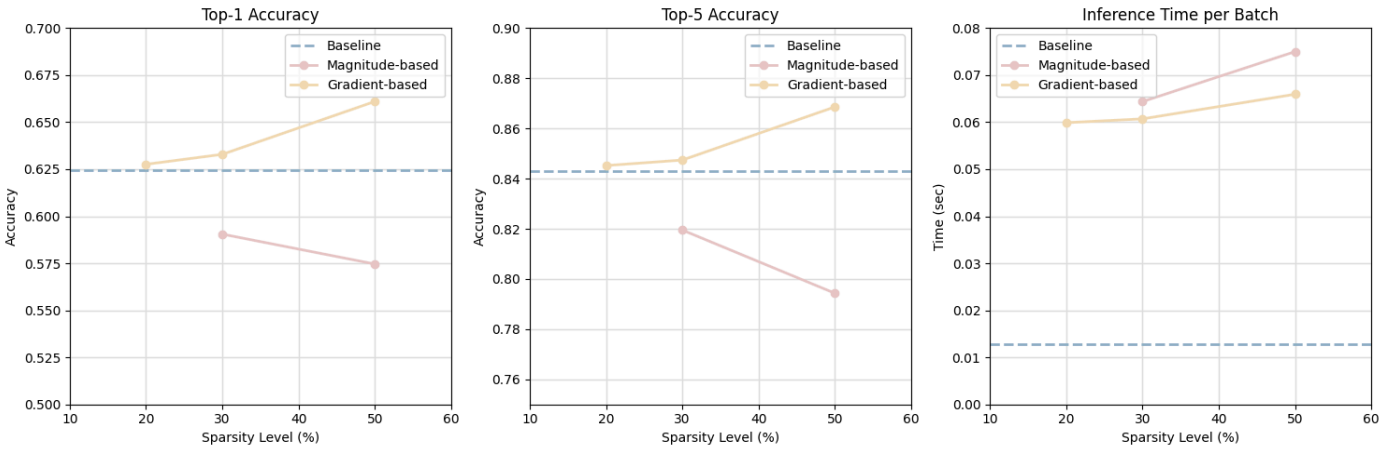


Figure 11: Top-1 Accuracy, Top-5 Accuracy, Inference Time per sample

**What stands out:**
Gradient-based pruning at 50% sparsity produced the best results:

- Top-1 Accuracy: 66.10% a ~4% improvement over baseline
- Top-5 Accuracy: 86.86%

This improvement is likely due to the overparameterized nature of ResNet-50 when applied to the relatively small-scale Tiny ImageNet-200 dataset. In such cases, pruning acts as an implicit form of regularization, enhancing generalization.

**Note :** Unstructured pruning creates sparsity by zeroing out insignificant weights. However, this does not translate into a significant reduction in model size unless sparsity-aware execution engines are used. As such, model size was not considered a primary metric for unstructured pruning in our evaluation.

**Conclusion :** Unstructured pruning proves to be beneficial for regularization in this context. Given the observed improvements in accuracy, we conclude that gradient-based pruning at 50% sparsity is the most effective unstructured pruning configuration for our ResNet-50 model on Tiny ImageNet-200.

## 4.2.3 Structured Pruning Findings

To reduce both computational load and model size, we applied **structured pruning** to ResNet-50 by removing entire channels (filters) from convolutional layers. We froze the initial convolutional block, the first layer of each residual block, and the fully connected (FC) layer, as these components are particularly sensitive to channel-wise pruning. Preserving these key layers helps retain essential representational capacity and prevents significant accuracy loss.

**What the table shows**: Top-1 accuracy, Top-5 accuracy, and model size at each structured pruning level, with percentage size reductions relative to the baseline.

| Sparsity Level | Top 1 Accuracy | Top 5 Accuracy | Model Size (MB) | Reduction in Model Size |
|---|---|---|---|---|
| 20% | 61.96% | 84.7% | 63.39 | 34% |
| 30% | 63.74% | 85.6% | 49.91 | 48% |
| 50% | 56.76% | 82.03% | 29.62 | 69% |

Table 1: Evaluation Metrics of Structured Pruning Models

**What stands out:** 30% sparsity delivered the best balance.

- Top-1 Accuracy improved to 63.74% (better than baseline).
- Model size dropped to 49.91 MB — a 48% reduction.

50% sparsity caused accuracy to drop (7%) sharply, falling below acceptable thresholds of (3%–5%). As such, this configuration was deemed unsuitable for further consideration in our compression pipeline.

**Conclusion** : Given its balance of accuracy preservation and significant compression, we selected the 30% structured pruning model as the starting point for our iterative pruning pipeline.

## 4.2.4 Iterative Pruning Findings

To explore whether combining structured and unstructured pruning would yield better compression-performance trade-offs, we implemented an **iterative pruning strategy**. This involved two stages:

1. **30% structured pruning**, followed by fine-tuning
2. **Gradient-based unstructured pruning** at either **20%** or **40%**, followed by another round of fine-tuning

**What stands out :** The gradient based pruning at 40% showed slightly better performance, improving the top-1 accuracy to **63.8%**, compared to **63.1%** from 20%. Notably, both configurations resulted in nearly identical model sizes (40%: **49.56 MB**, 20%: **49.89 MB**), reinforcing our selection of the 40% sparsity level for optimal trade-off.

**Conclusion :** Combining coarse (structured) and fine (unstructured) pruning steps helps eliminate redundant weights at multiple levels of granularity.

Gradient-based pruning is especially effective when the model has already been simplified through structured pruning. Early stopping avoids overfitting during second-stage fine-tuning, capturing performance gains efficiently**.**

## 4.2.5 Comparison of All Pruning Strategies

We experimented with three Pruning strategies. Unstructured pruning, Structured Pruning and Iterative Pruning. In this section we compare how the different strategies compare against each other on key evaluation metrics.

**What the plot shows**: Figure 12 compares structured, unstructured, and iterative pruning methods across key metrics: Top-1 accuracy, Top-5 accuracy, model size, and cross-entropy loss.

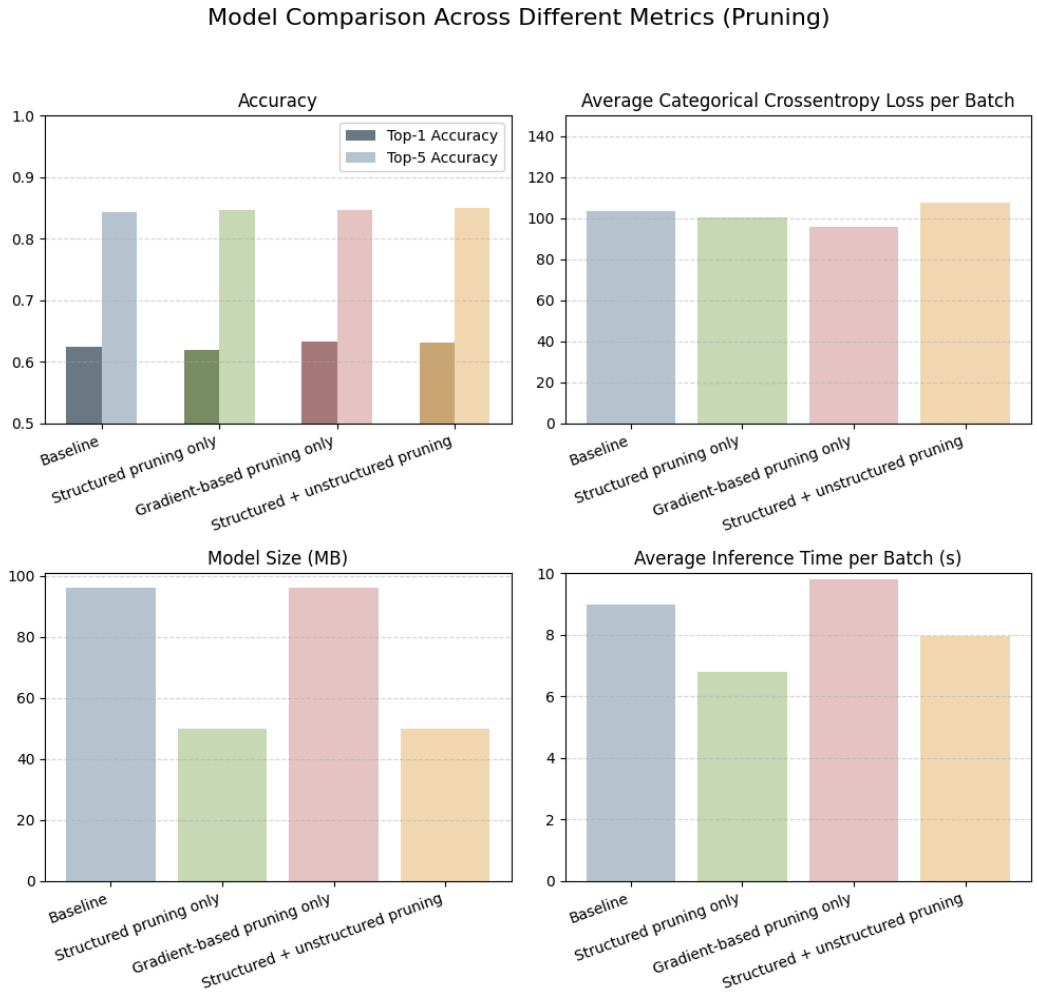


Figure 12: Model Comparison

**What Stood out** : Structured pruning alone proved highly effective for compressing model size while preserving accuracy. At 30% sparsity, the model not only reduced its size by 48% but also

exhibited a modest increase in both top-1 and top-5 accuracy, highlighting its ability to regularize the model without harming representational capacity.

**Unstructured pruning**, particularly the gradient-based variant, improved accuracy beyond the baseline. At 50% sparsity, it achieved the highest top-1 and top-5 accuracies across all strategies tested (66.10% and 86.86%, respectively). However, this did not significantly reduce model size, due to the limitations of unstructured sparsity in standard inference pipelines.
**Combining structured and unstructured pruning via an iterative approach** enabled us to leverage the benefits of both. Using structured pruning (30%) followed by gradient-based unstructured pruning (40%) resulted in a compressed model (49.56 MB) with enhanced accuracy (63.8% top-1), making it a balanced choice for practical deployment.

**Final Conclusions on pruning:** As shown in above figures, each pruning strategy introduces a distinct trade-off between accuracy, model size, and inference time. Structured pruning offers efficient compression with stability, while unstructured pruning (when sparsity-aware execution is enabled) offers improved performance. The iterative approach strikes an effective compromise, justifying its use as the preferred pruning strategy in our pipeline.

### 4.2.6 Quantization Findings

After pruning experiments, we explored quantization as a complementary compression method. The focus was on 8-bit integer quantization, applied post-training to avoid retraining overhead. Two types of quantization were tested:

- **Dynamic quantization**, which only quantized weights (typically applied to linear layers)
- **Static quantization**, which quantifies both weights and activations, and requires a calibration dataset

Quantization was applied both to the baseline model and to pruned models (specifically, the best iterative pruning configuration).

**What the graph shows**:
Figure 13 illustrate the impact of quantization (standalone and combined with pruning) across six model configurations:

1. Baseline
2. Baseline + Dynamic Quantization
3. Baseline + Static Quantization
4. Iterative Pruning
5. Iterative Pruning + Dynamic Quantization
6. Iterative Pruning + Static Quantization

Each configuration is evaluated for:

- Top-1 and Top-5 Accuracy
- Inference Time per Batch (seconds) (all inferences performed on CPU)
- Model Size (MB)

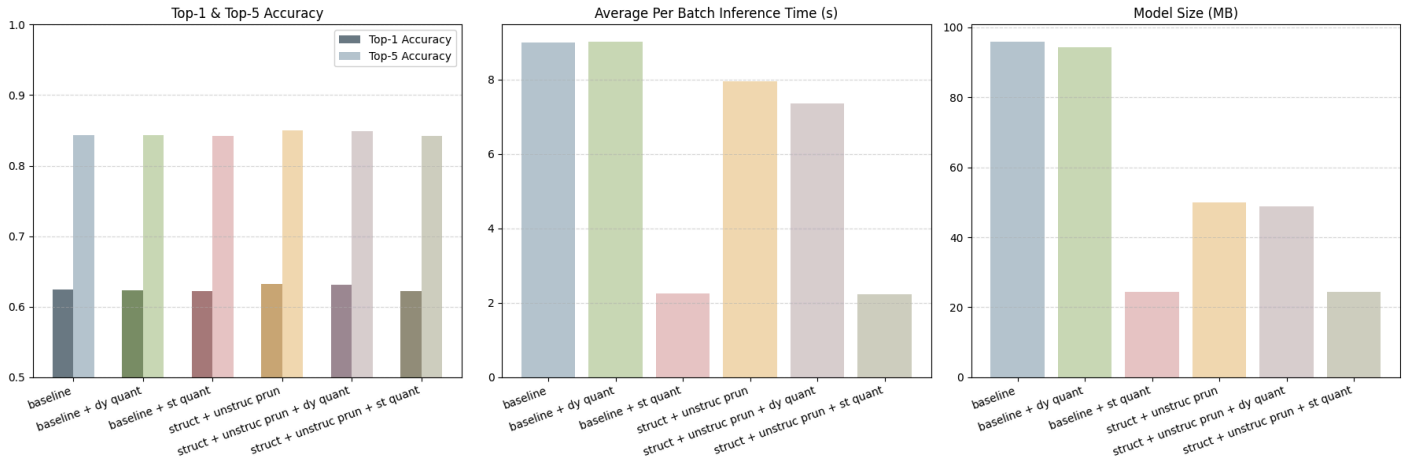


Figure 13: Quantization Findings

**What stands out:**

1. Accuracy was largely retained across all quantized variants, with only minimal drops. This is because we did per-channel quantization for resnet50 which helps preserve precision at the filter level. Additionally, the weights of Resnet50 are also nicely distributed which avoids a lot of clipping and hence, reduces quantization errors
2. Not much of a model size difference with dynamic quantisation because it has only one linear layer. Static Quantisation on the other hand had the lowest model size and the fastest inference. This is because Static quantization covers convolution layers (which dominate model size and computation), making it significantly more effective.
3. Quantisation boosted the inference speed of ResNet50. This is likely because ResNet-50 is a large model with a lot of computation. Converting operations from float32 to int8 reduced the overall computational load, making inference much faster.
4. Pruning plus quantization offers a better performance - speed and accuracy wise - than just pruning or just quantization. The best performing model is the pruned plus static quantized model. It has the lowest inference speed, though by a very small margin compared to baseline plus static quantized model.

# Chapter 5 : Future Scope

The project involved 1 week of planning and two weeks of execution, carried out entirely using our personal laptops and google colab. Here are some ideas we wish we could have explored if we had more time and GPU capabilities.

1. **Deeper Fine-Tuning of ResNet-50**:
   We modified ResNet-50 for Tiny ImageNet by replacing the output layer and unfreezing the last few layers for fine-tuning. With more time and compute, we could have fully fine-tuned the model. A better-trained model would likely have responded more effectively to pruning.
2. **Exploring Asymmetric and Alternative Quantization Schemes**:
   In addition to symmetric quantisation, we would also like to have our hands dirty with asymmetric quantisation. We would also like to experiment with Quantization-Aware Training (QAT) and see how that would improve the performance and to what degree.
3. **Scaling to Large Language Models (LLMs)**:
   In our free time, we would like to apply quantisation and pruning on Large Models like BERT. We would like to see how the compression techniques perform at scale. We wonder if strategies that worked with ResNet and LeNet would still be effective, or if an entirely different approach would be needed for models with millions of parameters.

# Chapter 6 : Conclusion

In this project explored the use of pruning and quantisation to compress two models of different scales. **LeNet on MNIST** and **ResNet-50 on Tiny ImageNet-200**. Our aim was to reduce model size and inference time while preserving accuracy — and in the process, understand the strengths and trade-offs of each technique.

## 6.1 Key Takeaways: Comparing LeNet and ResNet-50 Under Compression

- LeNet, being a small and shallow model, was already efficient. Compression led to massive size reductions but only marginal improvements in speed and almost no gains in accuracy.
- ResNet-50, on the other hand, is a large and overparameterized for Tiny ImageNet-200, which likely contributed to the increase in accuracy after pruning, particularly due to the regularization effect.
- Gradient-based unstructured pruning improved accuracy but reduced inference speed, as CPUs are not optimized for sparse matrix operations — masking weights without changing structure leads to computational inefficiency.
- In contrast, structured pruning removed entire filters, resulting in a model that was both faster and smaller, with a better balance between accuracy and deployment readiness.
- Quantization had a much more noticeable impact on ResNet-50 than on LeNet, especially in terms of model size and inference speed, due to its heavier architecture.
- The combination of iterative pruning + static quantization yielded the best overall results for ResNet-50, offering strong compression with minimal accuracy trade-off.

## 6.2 Final Reflection

Beyond just implementation, working on the project helped us better understand the tradeoffs and the decision making involved in model compression. We learned to balance accuracy, size and performance. We were able to get our feet wet with implementing various compression techniques, developing compression pipelines, comparing techniques, preparing models for deployment etc. Most importantly, we now have a deeper appreciation for how model structures and various compression strategies influence the accuracy and performance of models.

As a team, we had a lot of fun working together — sharing ideas, debating strategies, and learning from each other along the way. We would also like to sincerely thank Professor Gregory Provan for guiding our team and the entire class throughout this project.

***GitHub Repo Link:*** *https://github.com/lakshyasuri/ModelCompressionAnalysis/tree/master*